

Authentication Service: Project Documentation

1. Overview

This project implements a complete backend authentication system using Node.js and Express. It features secure user registration, login mechanisms, and protected route access. The system ensures that only authenticated users can access sensitive endpoints by utilizing JSON Web Tokens (JWT).

2. Technology Stack

The following libraries and technologies were used to build this service:

- **Node.js:** The Javascript runtime used to run the backend server.
- **Express.js:** A lightweight web framework used for routing HTTP requests and responses.
- **bcryptjs:** A cryptographic library used to securely hash and salt user passwords before they are stored.
- **jsonwebtoken (JWT):** Used to generate secure, stateless session tokens that are passed between the client and server.
- **dotenv:** Loads environment variables (like the JWT Secret Key) from a `.env` file to keep secrets out of the source code.

3. Step-by-Step Implementation Process

Step 1: Project Setup

- Initialized a new Node.js project using `npm init -y`.
- Installed the necessary dependencies: `npm install express bcryptjs jsonwebtoken dotenv`.
- Created a `.env` file to store the `JWT_SECRET` and `PORT`.
- Configured a `.gitignore` file to ensure the `.env` file and `node_modules` folder were not pushed to GitHub.

Step 2: In-Memory Storage Setup

- For the purpose of this assignment, a simple JavaScript array (`const users = []`) was utilized as an in-memory database to store registered users. This allows the application to run instantly without requiring a separate database setup.

Step 3: Registration Endpoint (POST /api/auth/register)

- Created an endpoint that accepts a `username` and `password` via the request body.
- Verified that the user does not already exist in the database.
- Used `bcrypt.genSalt` to generate a random salt, and `bcrypt.hash` to hash the user's plain-text password.
- Saved the new user object (containing the username and the hashed password) into the array.
- Returned a `201 Created` status upon success.

Step 4: Login Endpoint (POST /api/auth/login)

- Created an endpoint that accepts a `username` and `password`.
- Searched the database for the provided username. If not found, returned a `401 Unauthorized` error.
- Used `bcrypt.compare` to compare the provided plain-text password against the stored hashed password.
- Upon successful validation, a JSON Web Token (JWT) was signed using `jwt.sign()`. The token contains the user's identity and is set to expire in 1 hour.
- Sent the generated token back to the client.

Step 5: Authentication Middleware

- Created a custom Express middleware (`middleware/authenticate.js`).
- This function intercepts incoming requests to protected routes.
- It checks the HTTP headers for an `Authorization: Bearer <token>` string.
- If no token is provided, it immediately rejects the request with a `401 Unauthorized`.
- If a token is provided, it uses `jwt.verify()` to validate the token's authenticity against the server's secret key.
- If invalid or expired, it returns a `403 Forbidden`. If valid, it attaches the decoded user data to the request object and allows the request to proceed.

Step 6: Protected Route (GET /api/protected/secret)

- Created a route that specifically uses the authentication middleware.
- Because of the middleware, this route is completely isolated from unauthenticated users.
- When accessed successfully, it returns a personalized welcome message proving that the backend successfully identified the calling user.

4. Conclusion

By completing these steps, the backend successfully demonstrates how to securely manage user identities. The use of bcrypt ensures passwords are never stored in plain text, and JWTs allow the server to verify logged-in users without needing to constantly query a database or manage session cookies.